

HelixQA Integration Design (Phase 4 Follow-up — Lava-side)

Date: 2026-05-16 **Phase:** 4 follow-up (task #83 of constitution-compliance plan) **Status:** Design proposal — operator approval required before implementation **HelixQA pin:** 403603db (v4.0.0-256-g403603d) as of Phase 4 adoption commit aa0db6bd **Classification:** project-specific (the per-package integration choices are Lava-specific; the boundary-identification discipline is universal per HelixConstitution §11.4.31)

Context

Phase 4 (commit aa0db6bd) adopted HelixQA as submodules/helixqa at upstream HEAD with the HELIX_DEV_OWNED waiver pattern in 4 scanners. HelixQA is now PRESENT in the Lava tree but NOT INTEGRATED — no Lava code links against it; no Lava test invokes it; no Lava CI gate consumes it.

This document identifies WHICH parts of HelixQA add value to Lava + WHERE the integration boundary belongs.

HelixQA inventory (per submodules/helixqa/pkg/ survey)

HelixQA ships 30+ Go packages organized into these capability groups:

Group A — Directly applicable to Lava

Package	Purpose	Lava use case
pkg/ detector	Crash/ANR detection (ADB-based for Android, JVM/browser process monitoring)	Lava's Android client crash detection during Compose UI Challenge Tests + lava-api-go process monitoring
pkg/ evidence	Centralized screenshot, video, log, trace collection	Lava's Challenge Tests need screenshots when assertions fail (currently scattered across feature/*/test/)

Package	Purpose	Lava use case
pkg/ navigator	NavigationEngine with ADB / Playwright / X11 executors	Lava's Compose UI Challenge Tests drive UI via ADB on emulator — HelixQA's navigator is a richer abstraction
pkg/ validator	Step-by-step validation with per-step evidence	Lava's §6.J anti-bluff "primary assertion on user-visible state" pattern formalized
pkg/ ticket	Markdown issue ticket generation for AI fix pipelines	Lava's §6.O Crashlytics closure logs are conceptually adjacent — HelixQA's pattern could replace/extend

Group B — Conditionally applicable (need Lava use case to justify)

Package	Purpose	When Lava would need it
pkg/llm	LLM provider abstraction + adaptive fallback + cost tracking	If Lava adds LLM-driven test generation OR uses LLM-backed visual regression
pkg/ issuedetector	LLM-powered visual/UX/accessibility/functional bug detection	Same as above
pkg/ regression	Visual regression (SSIM-based + Lost Pixel / BackstopJS integration)	If Lava adopts pixel-diff Compose UI tests beyond the current text-assertion model
pkg/ autonomous	SessionCoordinator, PlatformWorker, PhaseManager	If Lava runs unattended overnight QA cycles
pkg/session	SessionRecorder, Timeline, VideoManager	If Lava starts recording video evidence per Challenge Test (currently screenshot-only via Espresso)

Group C — Not applicable to Lava

Package	Purpose	Why not applicable
pkg/discovery	Network host discovery (CPU/RAM/GPU capability scan)	Lava is single-host — no distributed-test fleet
pkg/distributed	Distributed test orchestration across multiple hosts	Same — Lava is single-host
pkg/gpu	GPU resource management for ML model inference	Lava's tests don't need ML model inference
pkg/audio	Audio capture + analysis	Lava is not an audio app (Boba is; Lava is torrent client)
pkg/capture	Real-time video capture pipeline	Out of scope per HelixQA's IMMEDIATE_EXECUTION_PLAN.md
pkg/gst	GStreamer integration	Same
pkg/learning, pkg/memory, pkg/planning	LLM-agent learning + memory + planning	Lava-side LLM agent is Claude Code (separate from HelixQA's LLM-agent infra)
pkg/agent, pkg/controller, pkg/maestro, pkg/nexus	LLM-driven autonomous test agents	Same
pkg/opensource, pkg/observe, pkg/performance	HelixQA-internal infra	Lava doesn't need to extend HelixQA
pkg/infra, pkg/bridge, pkg/bridges	HelixQA-internal integration shims	Same

Integration boundary options

Option 1 — Shell-level wiring only (LOWEST effort, RECOMMENDED for first cycle)

Wire HelixQA's existing 11 Challenge scripts into Lava's `scripts/run-challenge-matrix.sh` per §6.AE Comprehensive Challenge Coverage Mandate. HelixQA's scripts run inside Lava's existing emulator-matrix infrastructure.

Scope: ~150 LOC of bash glue + companion doc updates + sweep wrapper update.

HelixQA Challenge scripts integrated:

- `anchor_manifest_challenge.sh` — validates `docs/behavior-anchors.md` exists with anchors per user-visible capability
- `bluff_scanner_challenge.sh` — wraps `scripts/anti-bluff/bluff-scanner.sh` pattern scanner
- `chaos_failure_injection_challenge.sh` — chaos engineering harness
- `ddos_health_flood_challenge.sh` — DDoS-resilience health-endpoint flooding
- `host_no_auto_suspend_challenge.sh` — host-stability check (parallel to Lava's §6.M)
- `mutation_ratchet_challenge.sh` — go-mutesting kill-rate ratchet
- `no_suspend_calls_challenge.sh` — host-stability scan for suspend-call patterns (parallel to Lava's Host Machine Stability Directive)
- `scaling_horizontal_challenge.sh` — horizontal-scaling load test
- `stress_sustained_load_challenge.sh` — sustained-load stress test
- `ui_terminal_interaction_challenge.sh` — terminal UI interaction (limited Lava applicability)
- `ux_end_to_end_flow_challenge.sh` — end-to-end UX flow harness

§6.AE alignment: these 11 challenges are HelixQA's per-test-type harnesses; their integration into Lava's matrix runner satisfies §6.AE's "Comprehensive Challenge Coverage" mandate at the broader-than-Compose-UI layer.

Option 2 — Go-package linking for Group A packages (MEDIUM effort)

Lava-side Go code (`lava-api-go`) imports HelixQA's `pkg/detector` + `pkg/evidence` + `pkg/ticket` for use in `lava-api-go`'s own test infrastructure + crash-reporting pipeline.

Scope: lava-api-go's go.mod adds digital.vasic.helixqa as a dependency. Per HelixQA's README: go install digital.vasic.helixqa/cmd/helixqa@latest — the SDK is available. Lava-side Go code calls HelixQA APIs from new lava-api-go/internal/qa/ package.

Risk: HelixQA is in active development (per its README + IMMEDIATE_EXECUTION_PLAN.md). API stability is not guaranteed. Lava-side imports would couple to HelixQA's release cadence.

Mitigation: wrap every HelixQA API call in Lava-side adapter interfaces so version-bumps of HelixQA only touch the adapter layer.

FOLLOW-UP DESIGN: A detailed Phase 4-C design doc for Option 2 exists at docs/plans/2026-05-16-helixqa-go-package-linking-design.md (Phase 4 follow-up C, 2026-05-16). It covers: per-package public-API analysis + Lava-side adapter design + lava-api-go go.mod strategy + 4-phase rollout (4-C-1 pkg/evidence, 4-C-2 pkg/detector, 4-C-3 pkg/ticket, 4-C-4 pkg/navigator+pkg/validator) + \$6.J anti-bluff posture + 10 operator-blocking open questions. DESIGN-ONLY; no code changes yet; operator approval owed before Phase 4-C-1 starts.

Option 3 — Compose UI Challenge Test backend (HIGHEST effort)

Lava's existing Compose UI Challenge Tests under app/src/androidTest/kotlin/lava/app/challenges/ migrate to use HelixQA's pkg/navigator + pkg/evidence + pkg/ticket as backend services (via HelixQA CLI invocation OR via Go-package bridging from Kotlin via JNI / gRPC).

Scope: ALL 35+ Compose UI Challenge Tests get rewritten to use HelixQA as their backend. This is multi-week work.

Risk: HelixQA's navigator/evidence/ticket APIs are Go-only; bridging into Kotlin Android tests requires either subprocess invocation (slow) or JNI (complex) or gRPC (architecture overhead).

Recommendation: DEFER until Lava's existing Compose UI Challenge Tests show clear pain points the existing Espresso-based infrastructure can't solve.

Recommended path

This-cycle deliverable: Option 1 — Shell-level wiring of HelixQA's 11 Challenge scripts into Lava's scripts/run-challenge-matrix.sh. Single commit. No Go-package linking. No Compose UI test rewrites.

Future-cycle deliverable (Phase 4-follow-2): Option 2 — Go-package linking for Group A packages. Requires operator decision on: (a) lava-api-go vs the Android client as the integration entry-point, (b) adapter-interface design for HelixQA API stability isolation.

Out-of-scope (defer indefinitely): Option 3 — Compose UI Challenge Test backend migration. Re-evaluate only if Espresso-based tests prove insufficient.

§6.J anti-bluff posture

Any HelixQA integration MUST satisfy §6.J / §6.AE / §6.L:

1. Every HelixQA Challenge script wired into Lava's matrix MUST have a paired Lava-side falsifiability rehearsal (per §6.J.2) recorded in the wiring-commit's body.
2. Every HelixQA-backed assertion MUST be on user-visible state (per §6.J.3), not on HelixQA's internal data structures.
3. The wiring-commit MUST satisfy §6.AE.5 per-AVD attestation row format (no "wired but never executed" entries).
4. Any HelixQA API import MUST satisfy §6.AC non-fatal telemetry coverage (catch blocks call `recordNonFatal`).
5. Any HelixQA-driven CI gate MUST have a paired hermetic test (per Lava's existing tests/check-constitution/test_*.sh pattern).

Open questions for operator

1. **Container vs host runner:** HelixQA's Challenge scripts assume host execution. Lava's §6.X requires container-bound execution. Should the HelixQA challenges be wrapped to run inside submodules/containers/cmd/emulator-matrix --runner=containerized?
2. **Real-deps vs stub gating:** HelixQA's `bluff_scanner_challenge.sh` runs `go-mutesting` which requires Go toolchain + the project's Go modules. Should Lava's gate execute against `lava-api-go` (real) or stub the toolchain (mock)?
3. **Evidence directory boundary:** HelixQA emits evidence to its own `<helixqa>/evidence/<run-id>/`. Should Lava redirect to `.lava-ci-evidence/helixqa-challenges/<run-id>/?`
4. **§6.W mirror policy for HelixQA Challenge invocations:** HelixQA Challenge scripts may push artifacts or invoke external services. Lava's §6.W bounds remotes to GitHub + GitLab. Need audit of each script before wiring.

Status

DESIGN-ONLY. No code changes in this cycle. Operator approval owed before Option-1 wiring begins.

Cross-references

- `submodules/helixqa/README.md` — HelixQA framework overview
- `submodules/helixqa/docs/ANTI_BLUFF.md` — HelixQA's anti-bluff discipline (CONST-035 runbook)
- `submodules/helixqa/docs/architecture.md` — HelixQA package overview
- `submodules/helixqa/docs/IMMEDIATE_EXECUTION_PLAN.md` — HelixQA's 2026-04-08 Real-Time Video Pipeline roadmap
- `submodules/helixqa/docs/COMPREHENSIVE_VISION_INTEGRATION_PLAN.md` — HelixQA's 2026-04-09 CV integration plan
- `CLAUDE.md §6.AE` — Comprehensive Challenge Coverage + Container/QEMU Matrix Mandate
- `CLAUDE.md §6.J` — Anti-Bluff Functional Reality Mandate
- `CLAUDE.md §6.AC` — Comprehensive Non-Fatal Telemetry Mandate
- `docs/plans/2026-05-15-constitution-compliance.md` — parent constitution-compliance plan
- `docs/helix-constitution-gates.md` — gate index (HelixQA waivers documented per Phase 4)